



ACEnet Queuing

A Policy Modification Proposal

ACEnet Technical Team

December, 2007

Introduction

Now that the ACEnet environment has been operational for 1.5 years, certain issues with our N1 Grid Engine (N1GE) queuing implementation that impede effective use of the resources have been identified. These issues and solutions fall into four general categories:

1. Job Time Limits
2. Fairshare Queuing
3. Serial Subordination
4. Parallel Starvation
5. Interactive Work

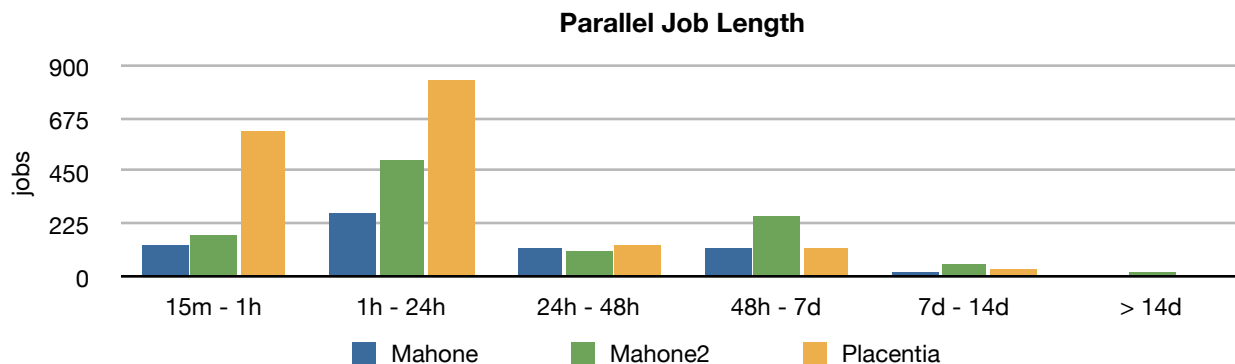
This discussion paper will outline the issues and put propose solutions. The essence of our solutions are indicated in **blue bold type**.

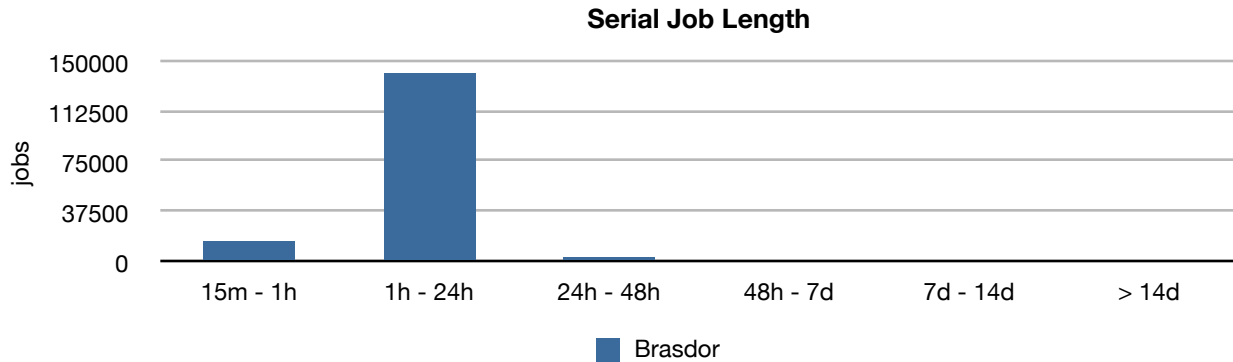
1. Job Time Limits

At the last CLUG meeting (Spring 2007), the Chairs made a motion requesting the ACEnet technical team to figure out how to impose a hard job time limit on our cluster queues. With no time limit, we are experiencing the following problems:

1. Users can take advantage of a lull in cluster activity to occupy a large number of cores indefinitely
2. There is nothing to encourage users to checkpoint their work potentially wasting CPU cycles if jobs die after running for a long time
3. Queue turn-over, at times, is quite slow, reducing the effectiveness of fairshare and frustrating deadline users

When exploring this issue, a number of users were encountered for whom a time limit would prove a hardship. In some cases their code doesn't have a checkpointing mechanism and, if it is code someone else has written, the user may not have the ability to add such a mechanism. In other cases the overhead of checkpointing large memory jobs is such that a high percentage of run time would be spent just checkpointing.





These two histograms show the distribution of job run lengths over the past year¹. The great majority of our jobs (81%) would fit easily into a 48 hour time limit, and indeed it is a relatively modest number of jobs that are causing the issues noted above. Note that the 19% of jobs over 48h consume much more than 19% of the CPU HRS.

With these caveats in mind, we propose the following:

Each cluster will have 2 queues, all.q and long.q with appropriate wallclock time limits as described in the following table.

When a soft run time limit is reached, the job receives a SIGTERM, which it can trap and use as a trigger to checkpoint. When the hard run time limit is reached, the job receives a SIGKILL and is immediately terminated.

all.q	long.q
includes all nodes on the cluster	contains only a modest % of the nodes on the cluster
has a hard run time limit of 48 hours and a soft run time limit of 47 hours	has a hard run time limit of 1 month and a soft run time limit of 1 months less 12 hours
the all.q nodes not also in long.q fill preferentially	

Over the past year, almost all jobs that exceeded 48 hours wallclock were no larger than 20 CPU. We propose to start with long.q on each cluster having access to something on the order of 24 cores and we would more tightly tune this as long jobs users are identified, and users incorporate checkpointing into their workflow. Our Computational Research Consultants would put in place a plan (seminars, scripts, tutorials, etc) to assist users in the transition to checkpointing.

Note that our intention is to make all.q and long.q compete with each other on the designated long.q nodes. This is to ensure that a lack of long jobs doesn't leave idle resources.

Under this plan, jobs would have the default time limits outlined above, but users are also able to more tightly specify their job lengths. If a user's job only runs an hour, there are benefits to letting the scheduler know this rather than assuming the 48 hour default (see backfilling in Section 4). As part of our queue education and document, we would encourage users to accurately specify their run times.

¹ Only jobs with wallclock greater than 15 minutes are considered here.

We believe this mechanism should largely address the issues noted and encourage checkpointing while accommodating our long run users.

For reference, WestGrid¹ and SHARCNET² have job time limits in place that range from 24h to 7d, depending on the machine.

2. Fairshare Queuing

Fairshare is an already implemented solution, thus this section contains no recommendation for change but rather explains the mechanism for determining priority in our existing environment. Fairshare addresses the problem of some users dominating the queue without imposing a complicated use accounting mechanism.

Currently, our queues are sorted by a fairshare algorithm that, in N1GE, is called a “project-based share tree”. It allots an equal share of resources for each ACEnet project. The system has “memory”. That is, jobs from projects that consumed fewer resources in the recent history are scheduled with priority over jobs from those projects that have been making heavier use of the resource. The duration of the memory has a half-life of 30 days in a decaying-weight scheme. This algorithm is intended to ensure that, over time, resources are equally distributed amongst the projects while not leaving resources idle if some projects aren’t consuming their share. Similarly, the scheduler will give an equal weighting to users within a single project.

The effect of fairshare is only on the ordering of jobs in queue. Running jobs are not interrupted.

Fairshare calculations do not span clusters.

3. Serial Subordination

Our current queue configuration on our parallel clusters have an all.q for regular parallel work and a subordinate serial queue to allow serial work to consume unused cycles on these clusters. While this remains an attractive notion, the practice has proven problematic in the following ways:

1. Our priority parallel queues have been kept quite busy resulting in some serial work remaining suspended (due to subordination) or queued for several months.
2. Suspended serial jobs occasionally continue to consume CPU cycles, resulting in an overloaded node. We have a Sun Support case open on this issue.
3. Suspended serial jobs continue to occupy the RAM they did while running, making this unavailable for the priority parallel jobs. Or more accurately, N1GE doesn't release the appropriate h_vmem consumable, effectively making the RAM unavailable.
4. Serial suspension isn't 1:1. Even a small parallel job will suspend all the serial jobs (up to 16) on the node it wants to run on.

However, we have heard from some users that the ability to do serial and parallel work on the same cluster is valuable. We propose to

remove the subordinate serial queues from our parallel clusters and instead accommodate serial work with the following measures

1. Add to the parallel clusters a killable.q. This queue would be subordinate to all other queues on the cluster and would only accept serial work. When a job in a superordinate queue needed one of the cores occupied by a killable.q job, the killable.q job would be terminated instead of suspended, thus this queue would be entirely non-blocking. There is an identified class of work for which this type of queue would be very well suited and it is felt this queue would ensure compute resources would not sit idle. Users would need to

¹ <http://www.westgrid.ca/support/quickstart>

² http://www.sharcnet.ca/Facilities/new_scheduling_directives.php

- request this queue explicitly. No one will land in it by accident.
2. We are in the process of adding 160 cores to our serial cluster, Brasdor, almost tripling its capacity to 252 cores.
 3. We propose to acquire for each of mahone, fundy, and placentia in our next round of purchases a modest number of x2200 nodes. We would dedicate these nodes to serial work. These nodes would fall under the 48 hour run time limit proposed above.

4. Parallel Starvation

Parallel starvation can occur when a mix of small and large job sizes are competing in the same queue. The N1GE scheduler wishes to fill idle resources as soon as they become available. If the highest priority job in the queue is too large to fit in the available resources, it will continue down (in priority) the queue until it finds a smaller job that will fit. This smaller, lower priority job will then be run ahead of the larger, higher priority job. Given a sufficient supply of small jobs, the larger jobs will languish (or “starve”) in the queue until, with luck, sufficient resources are freed all at once.

We have two proposals which, together, we expect will relieve the parallel starvation issue: **Big Job Days** and **Resource Reservation**.

Big Job Days

We propose to set aside 64 hours (end of business Friday to start of business Monday) once per month (or on a schedule determined by the RAC) when a cluster would be cleared of all running jobs and made available for jobs that would consume the entire cluster. Green resources would not be included here. We would ask that the RAC receive user proposals and allocate these Big Job Days.

Resource Reservation

Users are currently able to specify a flag to their queue request that would ask the scheduler to reserve resources. That is, no smaller jobs would jump ahead of them but rather, freed resources would sit idle until enough had accumulated to run the job. Only 5 jobs per cluster are permitted to be reserving resources at any given time.

Although currently implemented, resource reservation is not mentioned in our User Guide and we haven’t educated users about it. Our proposed action is to

describe resource reservation in the User Guide and on the user Wiki, and integrate it into new workshop and seminar offerings.

A disadvantage of resource reservation is the cycles it leaves idle while waiting for enough to become available to satisfy the large job’s request. However, if the scheduler were to know the run length of all jobs (as our first proposal would achieve) then it is able to backfill these idle cycles with short jobs that would complete before those resources would be called into service.

By default, the run time limit imposed under our proposal would be 48 hours, however, with backfilling, there is a motivation for users to even more tightly estimate the wallclock time of their job. We therefore propose to

describe backfilling and user-supplied time limits in the User Guide and on the user Wiki, and integrate it into new workshop and seminar offerings.

5. Interactive Work

We have encountered the following issues relating to interactive work that need to be addressed:

1. We have heard from users that debugging their work not only involves running their job interactively (which is currently done on the head nodes) but running it through the queuing system to identify problems in their submit scripts. The latter is hard to do if you have to wait hours before you find out your submit script has failed.
2. We have recently purchased the TotalView parallel debugger, which is used interactively. A user wouldn't want their debugging session queued and scheduled for some unknown time in the future.
3. Some users wish to run jobs (for debugging purposes) larger than the headnode can accommodate (over Myrinet, for example.)

NIGE does have the ability to schedule interactive sessions through the queue (qrsh, qsh, qlogin) however when these sessions are scheduled is dependent on the current mix of running and queued jobs and could result in quite a long wait. We propose to

create on all clusters a test.q which would have suspend priority over all other queues but would have a hard run time limit of one hour, and be limited to one job per user. Users would need to specifically request this queue and would not land in it by accident.

This is the solution SHARCNET has employed to deal with interactive work.